

# 대종사 십상을 웹의 가상 공간으로

원불교 대종사 십상과 관련 성지를  
웹에서 둘러보고 이해할 수 있는 공간을 준비합니다.

## 사진으로 담은 공간을 웹에서

성지를 촬영한 사진을 가우시안 스피클렛으로 변환해 공간의 모습을 담고,  
브라우저에서 걸으며 살펴보는 방식을 검토합니다.

## 공간 안에서 역사와 설명을 함께

역사·교법·장소 설명을 읽고, 과거의 건물이나 구조물을 3D 모델로 더해  
시대를 바꿔 살펴보는 경험을 염두에 두고 있습니다.

---

## 이 자료는 그에 필요한 웹 실행 기술의 시험 결과입니다

사전 기술 시험인 ‘스파이크’의 목적과 범위, 비교 결과를 정리했습니다.  
실행 앱에서는 같은 장면을 엔진별로 직접 열어볼 수 있습니다.

# 왜 먼저 작은 기술 시험을 했나

## 성지 촬영·제작 전에 가능성을 확인

공간을 띄우는 것뿐 아니라 바닥과 벽에 맞춰 걷기, 설명 열기, 시대 전환이 함께 동작해야 합니다.

대형 장면 로딩과 모바일 실행도 확인해 어떤 구성을 우선 검토할지 판단하려고 시험 앱을 만들었습니다.

## 특정 플랫폼에 되도록 묶이지 않도록

엔진이나 호스팅이 바뀌어도 공간 자산과 설명 데이터를 계속 활용하는 것이 목표입니다.

두 앱이 같은 Splat·GLB 파일과 공통 world.json을 읽습니다.  
PlayCanvas도 호스팅 Editor 대신 Engine을 코드로 사용합니다.

---

PlayCanvas와 Three.js를 같은 장면·기능으로 비교했습니다.

실행 앱은 정적 웹 파일이며, 현재 GitHub Pages에서 공유합니다.

엔진별 구현은 따로 필요합니다. 어떤 플랫폼으로든 수정 없이 옮길 수 있음을 검증한 결과는 아닙니다.

# 시험용 공간은 웹에서 내려받은 두 샘플

실제 성지를 촬영한 자료 대신 인터넷의 공개 샘플 두 개를 골랐습니다.  
대중사 십상과 관련된 장소를 보여주는 자료가 아니라 기능 시험용 공간입니다.

| 원본·제작자·출처                                                                                                                     | 사용 목적                                         |
|-------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Knock Community Hall · SuperSplat<br>SteveB(scbenoit)                                                                         | 기본 공간, 시대 전환, 구역 로딩<br>원본의 점 수를 줄이고 시험 장면을 만듦 |
| Sunnyvale Heritage Park Museum · SuperSplat<br>Damjan Cvetkov Dimitrov(zeitgeistarchivescans)<br>PlayCanvas 예제에서 배포한 SOG 다운로드 | 대형 원본 로딩, 보행, 벽 비침<br>분할·시대 전환 시험에도 사용        |

여러 실행 장면은 이 두 원본을 가공하거나 설정을 달리한 것입니다.  
‘과거 보기’의 건물·벽과 설명도 시험 데이터이며 실제 역사 복원 결과가 아닙니다.

## 이번에는 이미 만들어진 스플랫의 웹 실행을 시험했습니다

현장 촬영, 사진에서 스플랫 생성, 실제 역사 복원은 시험 범위에서 제외했습니다.

# 테스트 결과를 바탕으로 PlayCanvas + Ammo를 우선 추천

데스크톱에서 원본을 가장 빨리 불러왔고,  
iPhone 11에서도 대형 원본의 앱 준비 완료를 확인했다.

## 우선 후보

### PlayCanvas + Ammo

모바일에서도 대형 원본을 열고  
빠르게 시작하는 것이 중요할 때

## 대안

### Three.js + BVH

Three.js와의 연동을 우선하고  
점 수·SH를 줄일 수 있을 때

## 보류

### Rapier 기본 채택

현재 결과만으로 기본 구성을  
바꿀 근거는 부족

이번 스파이크 결과에 따른 조건부 추천이다. 최종 엔진은 아직 확정하지 않았다.

# 핵심 기능은 세 구성에서 동작했다

| 데스크톱 확인 항목                      | Three.js BVH | Three.js Rapier | PlayCanvas Ammo |
|---------------------------------|--------------|-----------------|-----------------|
| 시대별 화면·충돌 교체 / 겹침 시 안전한 위치로 이동  | 통과           | 통과              | 통과              |
| 전환 실패 → 이전 시대 복구 → 재시도          | 통과           | 통과              | 통과              |
| 구역별 상세 화면·충돌 데이터 로딩 / 해제 / 재로딩  | 통과           | 통과              | 통과              |
| 시대별 설명 표시 / 이전 설명 패널 정리         | 통과           | 통과              | 통과              |
| Sunnyvale 분할 로딩 + 시대 전환 + 설명 연동 | 통과           | 통과              | 통과              |

## 비교 구성

Three.js 0.186.1 / Spark 2.2.0  
BVH 0.9.15 또는 Rapier 0.21.0  
PlayCanvas 2.22.6 / Ammo

## 테스트 범위

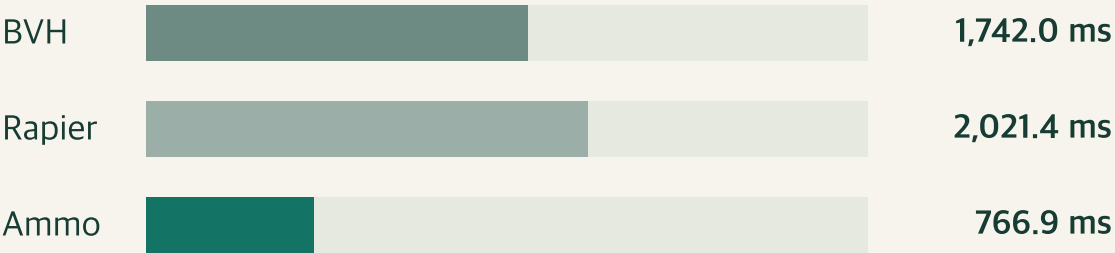
테스트용 시대별 벽과 설명으로 기능을 확인했다.  
실제 역사 복원이나 서비스 전체를 검증한 결과는 아니다.

# 같은 원본을 불러올 때 PlayCanvas가 가장 빨랐다

cold · 브라우저 HTTP 캐시 초기화



warm · 캐시를 유지하고 다시 로딩



| cold 전송량              | BVH      | Rapier   | Ammo     |
|-----------------------|----------|----------|----------|
| 페이지 + worker의 HTTP 요청 | 65.34 MB | 66.98 MB | 65.58 MB |

Sunnyvale 3,999,361점 SH3 · 조건별 3회(n=3)의 중앙값, 총 18회 · MB=1,000,000바이트

Apple M4 Pro / Chromium 154(headed) / ANGLE Metal / 앱·탭 각각 하나  
1200×863 CSS px, DPR 2(앱 상한 1.5) · localhost · 앱 준비 표시까지 측정, GPU가 첫 화면을 그린 시점과는 다름  
cold에서도 프로세스·OS 캐시는 유지했다. 물리 라이브러리만 비교한 수치는 아니다.

# 프레임 시간은 달랐지만 원인은 판단하기 어렵다

## 데스크톱 · Sunnyvale 원본

| 프레임 p95 (ms) | BVH  | Rapier | Ammo |
|--------------|------|--------|------|
| 정지           | 50.5 | 50.3   | 33.8 |
| 외벽 밀기        | 49.3 | 116.7  | 33.7 |

30초 구간별 p95를 3회 측정한 중앙값.  
렌더링·물리 계산·앱 실행 비용이 포함된다.  
접촉 위치·처리 방식이 달라 원인은 판단 보류.

## iPhone 11 · 소규모 Hall 장면

| 프레임 p95 (ms) | BVH | Rapier | Ammo |
|--------------|-----|--------|------|
| 정지           | 65  | 78     | 52   |

30초×3회 표본을 합쳐 계산한 p95.  
온도·순서·캐시 미통제, hasFocus=false.  
초기 이동은 바닥 밖으로 떨어져 비교에서 제외.

## 보행 계산 시간과 전체 프레임 시간은 다르다

데스크톱 보행 계산 시간의 p95 반복 중앙값은 0.1~0.6ms.  
Three.js의 앱 보행 step과 PlayCanvas의 rigidbody system은 포함 범위가 다르다.

# iPhone 대형 원본에서 가장 큰 차이가 나타났다

| Sunnyvale 데이터 / 구성          | 관찰          | 해석                                     |
|-----------------------------|-------------|----------------------------------------|
| 원본 SH3 / PlayCanvas Ammo    | 준비 완료 2회    | 모바일에서 원본 로딩 성공                         |
| 같은 원본 SH3 / Three.js BVH    | 준비 완료 못함    | 73.030초까지 디코딩·준비 완료 확인 못함 <sup>1</sup> |
| 같은 원본 SH3 / Three.js Rapier | 별도로 측정 안 함  | BVH 결과로 Rapier도 실패했다고 판단할 수 없음         |
| 같은 점 수 SH0 / Three.js BVH   | 47.320초에 준비 | 고차 SH를 제거한 비교 테스트 성공                   |
| 축소 SH0 분할 / 세 구성            | 기능 테스트 통과   | 원본과 품질·테스트 조건이 다름                      |

실제 iPhone 11 / iOS 27.0.1 (24A446) / Safari / Apple GPU  
<sup>1</sup> 추가 worker 진단에서는 2.250초에 다운로드 완료. 실패 원인·메모리 부족(OOM)은 미확정.

## 원본 로딩 성공과 부드러운 탐색은 별개다.

PlayCanvas 준비 완료까지 약 14.97초·19.11초. 캐시 조건을 맞춘 비교는 아니다.  
첫 탐색에는 약 1.57초 지연도 있었다. 축소본이 열려도 원본 SH3 문제가 해결된 것은 아니다.

# 분할 로딩은 가능했지만 원본과 품질이 다르다

900,000점 SH0 재인코딩 → 6개 구역 → 가까운 구역의 상세 화면·충돌 + 먼 구역의 간략한 화면

8.72 MB

시작할 때 선택한 데이터  
축소 분할본 전체 18.39MB

2 구역

동시에 유지한 상세 구역 최대값  
앱이 유지한 최대 점 수 448,028점

9 / 9

일반·실패 상황의 테스트 통과  
데스크톱 + iPhone, 세 구성

## 통과한 동작

준비 → 이동 → 해제 → 시작점 재로딩  
데스크톱 지연·실패·재시도 차단 처리  
원본 충돌면 유지, 경계 중복 약 +1.90%

## 결과를 볼 때 고려할 조건

400만 점 SH3 원본과 같은 품질은 아님  
52.6% 절감은 같은 축소 분할본 전체 대비  
유지한 점 수로 RAM 절감량을 알 수는 없음  
iPhone 입력은 합성 이벤트로 실행

추가 Sunnyvale 시대 전환·설명 연동은 데스크톱에서 통과. iPhone에서는 별도 검증이 필요하다.  
PlayCanvas의 sortKey overflow 경고 7회는 남아 있는 화면 품질 문제로 기록했다.

# 소규모 장면의 안정성과 메모리 측정 결과

Hall 약 10분 · 구성별 1회

앱 예외·context lost  
확인되지 않음

20×30초 구간 + 시대·설명 패널·충돌 표시 조작  
세 구성 모두 프레임 p95 17.4ms

PlayCanvas에서 favicon 404는 발생했다.  
대형 장면·모바일 장시간 안정성이나 메모리 누수가 없다는 근거는 아니다.

Sunnyvale warm · GC 실행 후 중앙값

| 메인 측정값 (MB)     | BVH    | Rapier | Ammo  |
|-----------------|--------|--------|-------|
| JS heap         | 7.43   | 12.34  | 8.46  |
| backing storage | 269.19 | 279.71 | 79.40 |

worker backing storage는 Three.js 약 0.053MB×3,  
PlayCanvas 84.19MB×1로 별도 측정.  
공유 버퍼가 중복될 수 있어 합계를 RAM으로 비교할 수 없다.  
GPU/VRAM·iPhone 메모리는 측정하지 않았다.

전체 메모리 사용량의 우열과 서비스 안정성은 판단을 보류한다.

## 벽 비침은 두 렌더러에서 공통으로 나타났다



원본 + 벽 뒤 진단 표식 · Three.js



같은 시점 + 벽 일부에 불투명 면 추가 · Three.js

PlayCanvas에서도 같은 현상을 확인했다. 선택한 픽셀 방향의 충돌 벽까지 거리는 1.644~1.809m, near는 0.05m였다. 이 현상을 가까운 영역의 잘림(near clipping)만으로 설명하기 어렵다.

추가한 면은 고정 시점에서 표식을 가렸지만, 가까이 가거나 아래를 보면 면의 경계와 질감 차이가 드러났다. 보정을 마친 결과물은 아니다. 원래 빨간 박스의 정확한 재현·가구 종류·단일 원인은 확인하지 못했다.

# 제작·수정 절차는 확인했지만 비용은 아직 모른다

| 작업        | 현재 작업 방식              | 측정하지 않았거나 남은 항목   |
|-----------|-----------------------|-------------------|
| 장면 데이터 교체 | SOG/GLB + 해시·좌표·충돌 검사 | 촬영·정합·품질 검수 시간    |
| 분할·LOD    | 공통 생성기 + 엔진별 생성·해제 처리 | 다른 장소에 적용·변경분만 생성 |
| 시대·설명     | JSON 작성 → 공통 생성·검사    | 실제 역사 자료·건물·바닥 제작 |
| 보행·벽 보정   | 공통 데이터 + 각 이동 제어기 검증  | 실측 치수·모바일 검수      |
| 배포        | 양쪽 앱에 공통 데이터 전체 복사    | 저장·전송 요금과 이용량     |

## 80.325초

Sunnyvale 데이터 18개를 생성한 프로그램 실행 시간.  
사람이 작업한 시간이나 인건비는 측정하지 않았다.

## 비용 순위는 만들지 않았다

현재 PlayCanvas는 호스팅 Editor를 쓰지 않는다.  
필요한 데이터만 로딩해도 배포 저장 공간은 줄지 않는다.

# 엔진 선택에 반영할 결과와 아직 확인하지 못한 항목

## 조건에 따른 추천

모바일 원본 로딩·빠른 시작

→ PlayCanvas + Ammo

Three.js 연동을 우선하고 데이터를 조정할 수 있으면

→ Three.js + BVH를 대안으로 유지

현재 근거만으로 Rapier를 기본 구성으로 채택하기는 어려움

## 아직 확인하지 못한 항목

대형 장면 모바일 장시간 안정성

실제 손가락 조작·Android

전체 RAM·GPU 메모리

실제 역사 자료·사람의 작업 시간·운영비

벽 비침은 두 구성의 공통 데이터·화면 품질 문제로 남긴다.

이번 보고서를 마무리하기 위해 추가로 수정할 필요는 없다.

---

## 근거를 다시 확인하는 방법

상세 Markdown: 수차·조건·빌드 버전·측정 기록 링크

발표 자료는 기존 테스트 기록을 요약했다. 문서 정리 중 성능을 다시 측정하거나 앱 소스를 바꾸지 않았다.

로딩 5135b39646a7 · 초기 모바일/안정성 6df18be3ac38 · 디코딩 비교 882e63d01db8

화면·충돌 분할 ee9a19b94a78 · 시대 전환·설명 연동 89a5df9eff9e · 서로 다른 빌드의 표본은 합치지 않았다.